TECHNICAL SEMINAR

10분 세미나
시멘틱 서치

의미 기반 검색의 구현과 최적화 전략

Embedding | Vector DB | Hybrid Search | Re-ranking

AGENDA

오늘 다룰 내용

시멘틱 서치의 핵심 개념부터 실전 최적화 전략까지

01
기초 개념
시멘틱 서치란? · 작동 원리

02
임베딩 & 평가
모델·차원 선택 · 재구축 비용 · 임계값 설계

03
인덱싱 & 인프라
KNN vs ANN · 필터링 전략 · 벡터 DB

04
한계 & 개선
시멘틱 서치의 한계 · 하이브리드 · 리랭킹

INTRODUCTION

시멘틱 서치란?

키워드 매칭을 넘어 의미를 이해하는 검색

🔤 전통적 검색

- 정확한 단어 매칭
- "자동차 수리" ≠ "차량 정비"
- BM25, TF-IDF 등

🧠 시멘틱 검색

- 문맥과 의미 이해
- "자동차 수리" ≈ "차량 정비"
- 벡터 임베딩 기반 유사도

ARCHITECTURE

작동 원리

문서는 미리 벡터화해 저장하고, 질의는 실시간으로 벡터화해 검색 Embedding-based retrieval

질의문 사용자 입력 → **임베딩 모델** 텍스트 → 벡터 → **유사도 계산** 코사인 유사도 → **결과 반환** 상위 K개

문서 임베딩 — 사전 처리

문서들 원본 데이터 → 임베딩 모델 텍스트 벡터화 → 벡터 수치 표현 → 벡터 DB 인덱스 저장

검색 실행 — 실시간 처리

질의문 사용자 입력 → 임베딩 모델 질의 벡터화 → 유사도 계산 가까운 후보 탐색 → 결과 top-k 반환

CORE COMPONENTS

모델과 차원 선택

모델과 차원은 처음 선택이 중요하다

관리형 범용

Gemini / OpenAI / Voyage

Gemini Embeddings, OpenAI text-embeddings, Voyage
API 기반이라 빠른 실험과 도입에 적합

오픈 범용

BGE / E5 / GTE 계열

자체 호스팅, 차원·비용 통제가 쉬움
운영 환경에 맞게 직접 튜닝하기 좋음

도메인 모델

파인튜닝 / 도메인 적응

전문 용어, 약어, 상품코드처럼 범용 모델이 자주 헷갈릴 때 검토
범용 모델로 기준선을 본 뒤 필요할 때 적용

▣ **차원도 신중하게 선택해야 한다** — 차원이 높을수록 벡터 공간이 넓어져 더 많은 정보를 담을 수 있지만, 메모리와 연산 비용도 함께 커진다.

! **모델이나 차원을 바꾸는 비용이 크다**
임베딩 모델이나 차원을 바꾸면 기존 벡터를 그대로 쓸 수 없어 **전체 데이터 재임베딩 + 인덱스 재구축 + 품질 재검증**이 필요하다

CHALLENGES

평가와 임계값 설계

후보 검색 평가는 랭킹 중심, 서빙 단계는 threshold도 함께 설계

⚠️ **완벽한 임계값은 없다**

- 서비스마다 다를 뿐 아니라, **같은 도메인 내에서도** 벡터 공간과 모델에 따라 임계값이 달라짐
- 임계값은 결과를 완벽하게 제어하는 수단이 아니라 **필터로 생각해야 함** — 노이즈를 걷어내는 용도이지 품질을 보장하지 않음
- threshold 하나보다 top-k 랭킹 품질 검증이 더 중요함

오프라인 평가

Recall@k, MRR, nDCG로 후보 검색 품질 확인

온라인 지표

CTR, zero-result rate, 재검색률로 운영 품질 추적

튜닝 원칙

threshold보다 top-k, chunking, rerank 조합을 함께 조정

💡 **왜 Recall@k / Precision@k를 봐야 하나:** 검색은 top-1이 아니라 top-k를 서빙하므로, 정답이 top-k 안에 포함되는지가 핵심이다. threshold 하나로 반환 여부를 자르기 전에 top-k 안에 타겟이 들어오는지부터 확인해야 함

INDEXING STRATEGY

KNN vs ANN 선택 전략

핵심 기준은 전체 규모보다 검색 시 실제 후보군 크기

KNN (Brute Force)

- ✓ 모든 벡터와 직접 비교
- ✓ Exact search라 recall 손실 없음
- ✓ 특히 pre-filter 후 후보군이 작을 때 가장 단순하고 정확

ANN (Approximate)

- ✓ 근사 알고리즘 (HNSW, IVF)
- ✓ 속도 우선, 약간의 정확도 손실
- ✓ 전역 검색이 크고 KNN이 실제로 느릴 때만 의미가 있음

💡 **핵심 인사이트: 후보군이 작다면 KNN이 충분히 빠를 수 있다**

ANN은 속도를 위해 recall을 일부 포기하는 선택이므로, 전역 검색이 크고 실제 탐색 비용이 커질 때 검토한다.
벡터 메모리 사용량은 차원에 비례해 증가하고, 실제 속도는 차원·구현체·필터 선택률에 따라 달라진다.

OPTIMIZATION

필터링 전략 선택

필터 선택률에 따라 KNN과 ANN 전략이 갈린다

전략 01

Pre-filter → KNN

필터 후 후보군이 소규모일 때 적합
완전 탐색, 구현 단순, 충분히 빠름

인덱스 불필요

전략 02

ANN → Post-filter

중대규모에서 검토
top-n 안에 필터 조건의 k개가 없으면 결과 부족 가능

결과 부족 주의

자세히 →

전략 03

Pre-filter → ANN

필터 후 부분 집합에 ANN을 적용하면 전체 데이터 기준으로 구축된 사전 인덱스가 깨져 성능 저하
엔진에 따라 아예 미지원

엔진 지원 확인

자세히 →

💡 **핵심은 필터 선택률** — 내 서비스·도메인에서 필터가 얼마나 효과적인지 먼저 고민해야 한다.
필터는 벡터 검색보다 **더 효과적이고 정확한** 후보 축소 수단이다.
선택률이 충분히 높으면 필터 후 KNN으로 끝나고, 그래도 후보군이 크게 남을 때만 ANN을 검토한다.
최근에는 **동적 인덱스**로 이 트레이드오프를 해결하려는 시도도 있다.

INFRASTRUCTURE

벡터 검색 인프라

제품명보다 기존 스택, 필터링, 하이브리드 지원, 운영 복잡도로 판단

전용 벡터 인프라

대규모 ANN, 멀티테넌시, 관리형 운영에 유리

Pinecone, Weaviate, Milvus 계열

기존 스택 확장

이미 운영 중인 DB/검색엔진과 통합이 쉬움

pgvector, Elasticsearch, MongoDB Atlas

저지연 / 캐시형

짧은 TTL, 실시간 응답, 캐시 계층에 적합

Redis 등 인메모리 계열

CHALLENGES

시멘틱 서치의 한계

실전에서 마주치는 주요 문제점들

키워드 매칭 약점
"2024년 1월" vs "2023년 1월"
고유명사, 제품번호 구분 어려움

도메인 지식 부족
범용 모델의 한계
전문 용어, 약어 이해 부족

연산 비용
대규모 데이터 리소스 소모
실시간 임베딩 생성 비용

💡 **한계를 느낀다면:**
• top-k 순위를 못 믿겠다 → **리랭킹** 도입으로 후보 재정렬 • 고유명사·키워드 매칭이 약하다 → **하이브리드 검색**으로 키워드 가중치 보완

⚠️ **그래도 남는 한계: 튜닝의 어려움**
파라미터에 정답이 없다 — 임계값·α·top-k는 서비스와 데이터마다 달라지며, 개선 효과를 측정할 피드백 데이터조차 모으기 어렵다.
이건 시멘틱 서치만의 문제가 아니라 **검색 시스템 전반의 본질적인 어려움**이다.

IMPROVEMENT STRATEGY

하이브리드 검색

시멘틱 + 키워드 검색의 장점을 결합

질의
사용자 입력

→

시멘틱 검색
의미 유사도

+

키워드 검색
BM25 / ES

→

결과 병합
정규화 + 가중치

$$\text{score} = \alpha \cdot \text{semantic_score} + (1-\alpha) \cdot \text{keyword_score}$$

α 가 클수록 의미 유사도 중심 · 작을수록 키워드 매칭 중심

⚠ **현실: α 값 튜닝도 매우 어렵다**

- 쿼리 유형별로 최적 결합 방식과 α 값이 다름
- 점수 스케일이 달라 정규화 없이 단순 합산하면 왜곡될 수 있음
- **완벽한 튜닝보다 로그 기반 반복 개선이 현실적**

ADVANCED STRATEGY

리랭킹 방식

후보 검색 뒤 더 정밀한 랭킹을 한 번 더 적용

1단계 후보 검색
벡터 / 하이브리드
많이 뽑고 빠르게 줄임

→

2단계 리랭킹
Cross-encoder 또는 LLM
정밀하게 다시 정렬

Cross-encoder 리랭킹

- 쿼리와 문서를 함께 넣어 관련도 점수 계산
- 전용 리랭커로 많이 쓰이는 실전 기본값
- LLM보다 저렴하고 지연시간 예측이 쉬움

LLM 리랭킹

- 문서 간 비교, 규칙 반영, 설명 생성까지 가능
- 복합 기준 판단에는 강하지만 비용과 지연이 큼
- 상위 소수 후보를 최종 정렬할 때 선택적으로 사용

시멘틱 서치 결과를 다시 정렬하는 단계 **후보를 좁힌 뒤 더 비싼 랭킹을 적용한다**

ARCHITECTURE

통합 아키텍처

한 화면에서 보는 실전 검색 파이프라인

쿼리 처리 분석 / 확장 → **하이브리드 검색** vector + BM25 → **병합 / 필터** 정규화 / 서비스 규칙 → **리랭커** 교차 인코더 / LLM → **최종 결과** top-k 서빙

! **튜닝 변수보다 데이터셋이 더 중요하다**
임계값 · 모델 · α 값을 조정하기 전에 **데이터셋의 벡터 공간 품질**을 먼저 봐야 한다 — 군집 구조에 따라 같은 파라미터도 완전히 다른 결과를 낸다

도입 자체는 비교적 현실적이다 — LLM 컨텍스트 축소 · 간단한 검색 용도라면 사이드 프로젝트에서도 충분히 시도할 수 있다. 단, **개선은 훨씬 어렵다** — 튜닝 포인트가 많고, 데이터 기반으로 효과를 검증하기도 쉽지 않다.

THANK YOU

Q&A
질문 환영합니다!

10분 세미나 시멘틱 서치

Embedding | Vector DB | Hybrid Search | Re-ranking

📚 **참고 자료** • 우아한 기술 블로그 • Do You Really Need a Vector Search Database? (fin.ai)